Universidad de Sevilla
Escuela Técnica Superior de Ingeniería Informática

Entregable 1: Motores de SQL para Big Data

Máster en Ingeniería del Software: Cloud, Datos y Gestión de las Tecnologías de la Información

Ingeniería de datos: Big Data

Curso 2025 – 2026

Autor	uvus	Correo
Jaime Linares Barrera	jailinbar	jailinbar@alum.us.es jlinares3@us.es

Índice de contenido

1. Introducción 2
1.1. Objetivo del entregable 2
1.2. Descripción del dataset 2
1.3. Tecnologías utilizadas 2
2. Arquitectura del sistema 4
2.1. Visión general 4
2.2. Componentes y roles 4
2.3. Comunicación entre componentes 5
3. Procedimiento 6
3.1. Paso 1: Ampliación del docker-compose con MinIO 6
3.2. Paso 2: Carga de informations_households.csv en PostgreSQL 7
3.3. Paso 3: Conversión del CSV a Parquet 9
3.4. Paso 4: Subida del Parquet a MinIO 10
3.5. Paso 5: Configuración de Trino 11
3.6. Paso 6: Registro del Parquet en Iceberg 14
4. Consultas federadas con Trino 17
4.1. Consumo diario por hogar 17
4.2. Consumo medio diario por grupo socioeconómico (Acorn) 18
4.3. Consumo medio diario por tipo de tarifa (Std vs ToU) 18
4.4. Consumo medio diario por mes y grupo socioeconómico 19
4.5. Top 10 hogares con mayor consumo total histórico 20
4.6. Consumo medio en invierno vs verano por grupo socioeconómico 21
4.7. Hogares sin datos de consumo en MinIO 22
4.8. Variabilidad de consumo diario por grupo socioeconómico 23
5. Conclusiones 24
5.1. Decisiones de arquitectura: de Hive a Iceberg 24
5.2. Compatibilidad de Trino 472 con Iceberg 24
5.3. Registro del Parquet en Iceberg 24
5.4. Reflexión final 25

1. Introducción

En esta sección se presenta el contexto y los fundamentos del entregable: el objetivo que se persigue, el dataset sobre el que se trabaja y las tecnologías empleadas para construir la arquitectura de análisis distribuido.

1.1. Objetivo del entregable

El presente ejercicio tiene como objetivo diseñar e implementar una arquitectura de análisis de datos distribuido que integre múltiples fuentes de datos heterogéneas, permitiendo realizar **consultas federadas** sobre ellas de forma unificada mediante **Trino**.

Para ello se han combinado dos tecnologías de almacenamiento con roles diferenciados: **PostgreSQL** como base de datos relacional para los datos descriptivos de los hogares, y **MinIO** como almacenamiento de objetos compatible con S3 para los datos de consumo energético en formato *Parquet*. Ambas fuentes se integran a través de **Trino**, que actúa como **motor de consulta distribuido**, permitiendo ejecutar JOINs federados entre datos almacenados en sistemas completamente distintos sin necesidad de moverlos ni transformarlos.

1.2. Descripción del dataset

Los datos utilizados provienen del dataset público *Smart Meters in London*, disponible en Kaggle. Este dataset recoge información sobre el consumo energético de hogares de Londres equipados con contadores inteligentes, y está compuesto por dos ficheros principales:

- ***informations_households.csv**:* contiene información descriptiva de cada hogar, incluyendo su identificador único (LCLid), el tipo de tarifa contratada (stdorToU), y su clasificación socioeconómica según el sistema ACORN. Este fichero contiene 5.566 hogares y se carga en PostgreSQL.
- ***daily_dataset.csv**:* contiene los datos de consumo diario agregado por hogar, con métricas como la suma, media, mediana, máximo, mínimo y desviación estándar del consumo energético para cada día. Cuenta con más de 3,5 millones de registros y se almacena en MinIO en formato Parquet.

1.3. Tecnologías utilizadas

La arquitectura desplegada hace uso de las siguientes tecnologías, todas ellas orquestadas mediante Docker Compose:

- **Trino**: motor de consulta SQL distribuido y federado. Permite consultar simultáneamente múltiples fuentes de datos heterogéneas usando SQL estándar. Actúa como punto central de la arquitectura.
- **PostgreSQL**: sistema de gestión de bases de datos relacional donde se almacenan los datos descriptivos de los hogares. Se accede desde Trino mediante el conector nativo de PostgreSQL.

- **MinIO**: sistema de almacenamiento de objetos compatible con la API de Amazon S3. Almacena el fichero Parquet con los datos de consumo diario, actuando como Data Lake de la arquitectura.
- **Apache Iceberg**: formato de tabla abierto para grandes conjuntos de datos analíticos. Permite a Trino consultar los ficheros Parquet almacenados en MinIO como si fueran tablas SQL, gestionando los metadatos necesarios para ello.
- **Iceberg REST Catalog**: servicio que implementa la especificación REST de catálogo de Iceberg. Gestiona los metadatos de las tablas Iceberg y sirve de intermediario entre Trino y MinIO.
- **PyIceberg**: librería Python que implementa el cliente de Apache Iceberg. Se utiliza para registrar el fichero Parquet existente en MinIO como una tabla Iceberg, generando los metadatos necesarios para que Trino pueda consultarla.

2. Arquitectura del sistema

En esta sección se describe la arquitectura desplegada, los componentes que la forman y cómo se comunican entre sí.

2.1. Visión general

La arquitectura implementada sigue el patrón de **Data Lakehouse federado**: combina un Data Lake basado en almacenamiento de objetos (MinIO) con una base de datos relacional (PostgreSQL), exponiendo ambas fuentes a través de un único motor de consulta federado (Trino). Esto permite ejecutar consultas SQL que combinan datos de ambos sistemas sin necesidad de replicarlos ni centralizarlos.

El flujo de datos es el siguiente: los datos descriptivos de los hogares se almacenan en PostgreSQL, mientras que los datos de consumo diario se almacenan como fichero Parquet en MinIO y se registran en Apache Iceberg mediante PyIceberg. Trino, configurado con dos catálogos (postgres e iceberg), puede acceder a ambas fuentes simultáneamente y ejecutar JOINs federados entre ellas.

2.2. Componentes y roles

La arquitectura está compuesta por cuatro servicios, todos ellos desplegados como contenedores Docker dentro de la misma red **trino_network**:

- **Trino**: actúa como capa de consulta central. Recibe las consultas SQL del usuario, las descompone y las ejecuta en paralelo contra las fuentes de datos correspondientes. Expone su interfaz en el puerto 8080.
- **PostgreSQL**: almacena la tabla households con los datos descriptivos de los 5.566 hogares. Trino accede a él a través del catálogo postgres, configurado con el conector JDBC nativo. Expone su interfaz en el puerto 5432.
- **MinIO**: actúa como Data Lake, almacenando el fichero daily_dataset.parquet en el bucket smart-meter. Es compatible con la API de Amazon S3, lo que permite que tanto Trino como PyIceberg accedan a él usando credenciales y endpoints S3 estándar. Expone su API en el puerto 9000 y su web en el 9001.
- **Iceberg REST Catalog**: servicio que gestiona los metadatos de las tablas Iceberg. Cuando Trino necesita consultar una tabla Iceberg, pregunta al REST Catalog dónde están los datos y cómo está estructurada la tabla. Expone su API en el puerto 8181.

También tenemos **PyIceberg**, que es una librería Python ejecutada localmente. Se utiliza en el paso de carga de datos para registrar el fichero Parquet de MinIO como una tabla Iceberg en el REST Catalog, generando los metadatos de esquema y localización necesarios.

2.3. Comunicación entre componentes

Todos los servicios comparten la red Docker **trino_network**, lo que les permite comunicarse entre sí usando sus nombres de contenedor como hostname. Las relaciones principales son:

- **Trino → PostgreSQL**: mediante JDBC, a través del catálogo postgres configurado en *trino/conf/catalog/postgres.properties*.
- **Trino → Iceberg REST Catalog**: mediante HTTP, para resolver los metadatos de las tablas Iceberg.
- **Trino → MinIO**: mediante el protocolo S3, para leer los ficheros Parquet físicos una vez que conoce su ubicación a través del REST Catalog.
- **Iceberg REST Catalog → MinIO**: mediante el protocolo S3, para leer y escribir los metadatos de las tablas Iceberg.
- **PyIceberg → REST Catalog**: mediante HTTP, para registrar la tabla y sus metadatos.
- **PyIceberg → MinIO**: mediante el protocolo S3, para leer el esquema del fichero Parquet y registrar los ficheros de datos.

3. Procedimiento

En esta sección se detalla el proceso completo seguido para construir la arquitectura, desde la configuración inicial del entorno hasta el paso previo a la ejecución de las consultas federadas. Cada paso se describe de forma que el proceso sea reproducible.

3.1. Paso 1: Ampliación del docker-compose con MinIO

El entorno base contaba ya con tres servicios: MySQL, PostgreSQL y Trino. En este primer paso se amplió el fichero *docker-compose.yml* añadiendo el servicio **MinIO**, que actuará como almacenamiento de objetos compatible con S3 a lo largo de todo el entregable.

Los cambios realizados sobre el fichero base fueron tres.

En primer lugar, se añadió el volumen persistente minio-data en la sección volumes del fichero, para garantizar que los datos almacenados en MinIO se conserven entre reinicios o recreaciones del contenedor.

```yaml
volumes:
  mysql-data:
  pg-data:
  minio-data:
```

En segundo lugar, se añadió el bloque de configuración del servicio MinIO. Se configura con usuario y contraseña (minio / minio123), expone el puerto 9000 para la API S3 y el 9001 para la interfaz web, y monta el volumen minio-data para persistir los datos.

```yaml
# ==========================================================
# MinIO
# ==========================================================
▷Run Service
minio:
  image: minio/minio:latest
  container_name: minio
  hostname: minio
  restart: unless-stopped
  environment:
    MINIO_ROOT_USER: minio
    MINIO_ROOT_PASSWORD: minio123
  command: server /data --console-address ":9001"
  ports:
    - "9000:9000"
    - "9001:9001"
  volumes:
    - minio-data:/data
  networks:
    - trino_network
```

En tercer lugar, se actualizó el *depends_on* del servicio Trino para incluir minio, garantizando que MinIO esté disponible antes de que Trino arranque.

```yaml
# ==================================================
# Trino
# ==================================================
▷Run Service
trino:
  image: trinodb/trino:472
  container_name: trino
  hostname: trino
  restart: unless-stopped
  user: root
  environment:
    - TRINO_CONFIG_DIR=/etc/trino
  volumes:
    - ./trino/conf:/etc/trino
    - ./trino/conf/catalog:/etc/trino/catalog
  depends_on:
    - mysql
    - postgres
    - minio
  ports:
    - "8080:8080"
  networks:
    - trino_network
```

Una vez realizados estos cambios, se levantó el entorno completo.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker compose up -d
time="2026-03-21T15:27:01+01:00" level=warning msg="C:\\Users\\jaime\\Desktop\\BD (Big Data)\\Ecosistema Hadoop\\Teoría\\202526-S2-BigDataSQL\\docker-compose.yml: the attribute `version` is ob
solete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 46/46
 ✓ mysql Pulled                                                                                                                                                                    32.2s
 ✓ minio Pulled                                                                                                                                                                    20.0s
 ✓ postgres Pulled                                                                                                                                                                 28.3s
 ✓ trino Pulled                                                                                                                                                                    62.5s
[+] Running 8/8
 ✓ Network 202526-s2-bigdatasql_trino_network  Created                                                                                                                              0.0s
 ✓ Volume 202526-s2-bigdatasql_pg-data         Created                                                                                                                              0.0s
 ✓ Volume 202526-s2-bigdatasql_minio-data      Created                                                                                                                              0.0s
 ✓ Volume 202526-s2-bigdatasql_mysql-data      Created                                                                                                                              0.0s
 ✓ Container minio                             Started                                                                                                                              1.7s
 ✓ Container mysql                             Started                                                                                                                              1.7s
 ✓ Container postgres                          Started                                                                                                                              1.7s
 ✓ Container trino                             Started                                                                                                                              0.9s
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS                          PORTS                                                   NAMES
d1e7b8f99470   trinodb/trino:472     "/usr/lib/trino/bin/…"   24 seconds ago   Up 23 seconds (health: starting)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp           trino
08d4d977db94   postgres:15           "docker-entrypoint.s…"   25 seconds ago   Up 23 seconds                   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp             postgres
b327548b8a86   minio/minio:latest    "/usr/bin/docker-ent…"   25 seconds ago   Up 23 seconds                   0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp  minio
96dc08d5d444   mysql:8.0             "docker-entrypoint.s…"   25 seconds ago   Up 23 seconds                   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp             mysql
```

3.2. Paso 2: Carga de informations_households.csv en PostgreSQL

El objetivo de este paso es preparar la primera fuente de datos de la arquitectura: cargar la información descriptiva de los hogares en PostgreSQL, donde Trino podrá consultarla a través del catálogo postgres.

El proceso se divide en tres partes: creación de la base de datos y la tabla, carga del CSV y verificación de los datos.

1) Creación de la base de datos y la tabla

Se crea el script *postgres/create_households.sql* con la estructura necesaria. En primer lugar se crea la base de datos households_db, se conecta a ella y se define la tabla households con las columnas correspondientes a los campos del CSV: LCLid como clave primaria, stdorToU, Acorn, Acorn_grouped y file.

```
postgres > create_households.sql
CREATE DATABASE households_db;

\c households_db

CREATE TABLE households (
    LCLid TEXT PRIMARY KEY,
    stdorToU TEXT,
    Acorn TEXT,
    Acorn_grouped TEXT,
    file TEXT
);
```

El script se ejecuta desde PowerShell pasándolo directamente al contenedor de PostgreSQL.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> Get-Content .\postgres\create_households.sql | docker exec -i postgres psql -U demo
CREATE DATABASE
You are now connected to database "households_db" as user "demo".
CREATE TABLE
```

2) Carga del CSV

Con la tabla ya creada, se carga el fichero informations_households.csv directamente desde PowerShell usando el comando COPY de PostgreSQL, que permite importar ficheros CSV de forma eficiente.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> Get-Content .\data\informations_households.csv | docker exec -i postgres psql -U demo -d
 households_db -c "COPY households (LCLid, stdorToU, Acorn, Acorn_grouped, file) FROM STDIN WITH (FORMAT csv, HEADER true, DELIMITER ',');"
COPY 5566
```

3) Verificación

Se accede al CLI de PostgreSQL para comprobar que los datos se han cargado correctamente.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker exec -it postgres psql -U demo -d households_db
psql (15.17 (Debian 15.17-1.pgdg13+1))
Type "help" for help.

households_db=# SELECT COUNT(*) FROM households;
 count
-------
  5566
(1 row)

households_db=# SELECT * FROM households LIMIT 10;
   lclid   | stdortou |  acorn  | acorn_grouped |  file
-----------+----------+---------+---------------+---------
 MAC005492 | ToU      | ACORN-  | ACORN-        | block_0
 MAC001074 | ToU      | ACORN-  | ACORN-        | block_0
 MAC000002 | Std      | ACORN-A | Affluent      | block_0
 MAC003613 | Std      | ACORN-A | Affluent      | block_0
 MAC003597 | Std      | ACORN-A | Affluent      | block_0
 MAC003579 | Std      | ACORN-A | Affluent      | block_0
 MAC003566 | Std      | ACORN-A | Affluent      | block_0
 MAC003557 | Std      | ACORN-A | Affluent      | block_0
 MAC003553 | Std      | ACORN-A | Affluent      | block_0
 MAC003482 | Std      | ACORN-A | Affluent      | block_0
(10 rows)
```

El resultado confirma que los 5.566 hogares se han cargado correctamente en PostgreSQL.

3.3. Paso 3: Conversión del CSV a Parquet

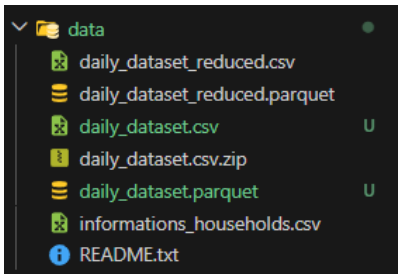
El objetivo de este paso es transformar el fichero de consumo diario *daily_dataset.csv* al formato **Parquet**, que es el formato de almacenamiento columnar estándar en arquitecturas de Data Lake. Parquet ofrece ventajas significativas frente al CSV en términos de compresión y rendimiento en consultas analíticas, al permitir leer únicamente las columnas necesarias sin cargar el fichero completo en memoria.

Para realizar la conversión se crea el script *src/csv_to_parquet.py*, que utiliza la librería **pandas** para leer el CSV y escribirlo directamente en formato Parquet:

```
src > csv_to_parquet.py > ...
import pandas as pd

df = pd.read_csv("data/daily_dataset.csv")
df.to_parquet("data/daily_dataset.parquet", index=False)

print("Archivo Parquet creado correctamente: data/daily_dataset.parquet")
print(df.head())
print(df.dtypes)
```

El script se ejecuta desde PowerShell:

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> python .\src\csv_to_parquet.py
Archivo Parquet creado correctamente: data/daily_dataset.parquet
       LCLid         day  energy_median  energy_mean  energy_max  energy_count  energy_std  energy_sum  energy_min
0  MAC000131  2011-12-15         0.4850     0.432045       0.868            22    0.239146       9.505       0.072
1  MAC000131  2011-12-16         0.1415     0.296167       1.116            48    0.281471      14.216       0.031
2  MAC000131  2011-12-17         0.1015     0.189812       0.685            48    0.188405       9.111       0.064
3  MAC000131  2011-12-18         0.1140     0.218979       0.676            48    0.202919      10.511       0.065
4  MAC000131  2011-12-19         0.1910     0.325979       0.788            48    0.259205      15.647       0.066
LCLid             object
day               object
energy_median    float64
energy_mean      float64
energy_max       float64
energy_count       int64
energy_std       float64
energy_sum       float64
energy_min       float64
dtype: object
```



La ejecución confirma que el fichero *data/daily_dataset.parquet* se ha generado correctamente. El dataset resultante contiene las siguientes columnas:

- *LCLid*: identificador del hogar, de tipo texto.
- *day*: fecha del registro, de tipo texto.
- *energy_median*, *energy_mean*, *energy_max*, *energy_std*, *energy_sum*, *energy_min*: métricas de consumo energético diario, todas de tipo float64.

- *energy_count*: número de mediciones realizadas ese día, de tipo int64.

3.4. Paso 4: Subida del Parquet a MinIO

El objetivo de este paso es subir el fichero *daily_dataset.parquet* al bucket *smart-meter* de MinIO, donde quedará disponible como fuente de datos para Iceberg y Trino.

Para interactuar con MinIO desde la línea de comandos se utiliza el cliente MinIO Client (*mc*), ejecutado como contenedor Docker para no requerir instalación local. El proceso se divide en cuatro partes.

1) Preparación del cliente mc

En primer lugar se crea una carpeta local .mc para almacenar la configuración del cliente y se configura el alias local apuntando al servidor MinIO desplegado en el contenedor.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> mkdir .mc


    Directorio: C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        21/03/2026     18:04                .mc


PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker run --rm -v "${PWD}\.mc:/root/.mc" --network 202526-s2-bigdatasql_trino_network min
io/mc alias set local http://minio:9000 minio minio123
Unable to find image 'minio/mc:latest' locally
latest: Pulling from minio/mc
b7c1efd1d2b4: Pull complete
9404e0df6bd3: Pull complete
4a93952ab9ce: Pull complete
b268f447c871: Pull complete
f29310fc4a8a: Pull complete
Digest: sha256:a7fe349ef4bd8521fb8497f55c6042871b2ae640607cf99d9bede5e9bdf11727
Status: Downloaded newer image for minio/mc:latest
Added `local` successfully.
```

2) Creación del bucket

Se crea el bucket smart-meter, que actuará como contenedor principal de los datos del Data Lake.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker run --rm -v "${PWD}\.mc:/root/.mc" --network 202526-s2-bigdatasql_trino_network min
io/mc mb local/smart-meter
Bucket created successfully `local/smart-meter`.
```

3) Subida del fichero Parquet

Se sube el fichero daily_dataset.parquet al bucket recién creado.

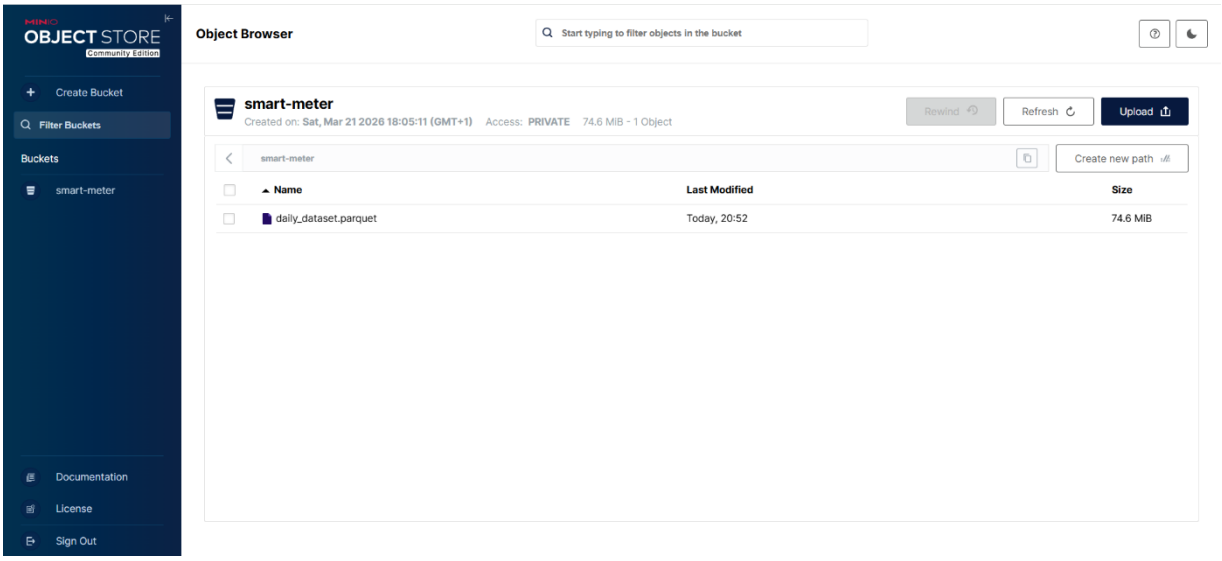
```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker run --rm -v "${PWD}\.mc:/root/.mc" -v "${PWD}\data:/data" --network 202526-s2-bigda
tasql_trino_network minio/mc cp /data/daily_dataset.parquet local/smart-meter/
`/data/daily_dataset.parquet` -> `local/smart-meter/daily_dataset.parquet`

| Total | Transferred | Duration | Speed |
|---|---|---|---|
| 74.61 MiB | 74.61 MiB | 00m00s | 154.38 MiB/s |
```

4) Verificación

Se verifica que el fichero se ha subido correctamente listando el contenido del bucket.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker run --rm -v "${PWD}\.mc:/root/.mc" --network 202526-s2-bigdatasql_trino_network min
io/mc ls local/smart-meter
[2026-03-21 17:05:23 UTC]  75MiB STANDARD daily_dataset.parquet
```

Opcionalmente, se puede comprobar también desde la consola web de MinIO accediendo a http://localhost:9001 con las credenciales minio / minio123.



3.5. Paso 5: Configuración de Trino

El objetivo de este paso es configurar Trino para que pueda acceder a las dos fuentes de datos de la arquitectura: PostgreSQL, donde se encuentran los datos descriptivos de los hogares, y MinIO, donde se almacena el fichero Parquet de consumo diario a través de Iceberg.

El proceso se divide en tres partes: configuración del catálogo PostgreSQL, incorporación del Iceberg REST Catalog al entorno y configuración del catálogo Iceberg.

1) <u>Configuración del catálogo PostgreSQL</u>

Se modifica el fichero *trino/conf/catalog/postgres.properties*, que indica a Trino cómo conectarse a la base de datos households_db de PostgreSQL mediante JDBC.

```
trino > conf > catalog > postgres.properties
connector.name=postgresql
connection-url=jdbc:postgresql://postgres:5432/households_db
connection-user=demo
connection-password=demo123
```

```
trino> SHOW TABLES FROM postgres.public;
   Table
------------
 households
(1 row)

Query 20260321_184531_00001_n3qrn, FINISHED, 1 node
Splits: 19 total, 19 done (100.00%)
0.51 [1 rows, 26B] [1 rows/s, 51B/s]

trino> SELECT * FROM postgres.public.households LIMIT 5;
   lclid    | stdortou |  acorn   | acorn_grouped |  file
-----------+----------+---------+---------------+---------
 MAC005492 | ToU      | ACORN-  | ACORN-        | block_0
 MAC001074 | ToU      | ACORN-  | ACORN-        | block_0
 MAC000002 | Std      | ACORN-A | Affluent      | block_0
 MAC003613 | Std      | ACORN-A | Affluent      | block_0
 MAC003597 | Std      | ACORN-A | Affluent      | block_0
(5 rows)

Query 20260321_184555_00002_n3qrn, FINISHED, 1 node
Splits: 1 total, 1 done (100.00%)
0.29 [5 rows, 0B] [17 rows/s, 0B/s]
```

2) Incorporación del Iceberg REST Catalog

Para que Trino pueda consultar tablas Iceberg almacenadas en MinIO, necesita un servicio que gestione los metadatos de dichas tablas. Se elige el **Iceberg REST Catalog** por su simplicidad de configuración y su compatibilidad con el stack desplegado.

Se añade el servicio *iceberg-rest* al fichero *docker-compose.yml*. Este servicio se configura apuntando al bucket s3://smart-meter/ como warehouse y recibe las credenciales de acceso a MinIO como variables de entorno.

```yaml
# ==========================================================
# Iceberg REST Catalog
# ==========================================================
▷Run Service
iceberg-rest:
  image: tabulario/iceberg-rest:latest
  container_name: iceberg-rest
  hostname: iceberg-rest
  restart: unless-stopped
  environment:
    CATALOG_WAREHOUSE: s3://smart-meter/
    CATALOG_IO__IMPL: org.apache.iceberg.aws.s3.S3FileIO
    CATALOG_S3_ENDPOINT: http://minio:9000
    CATALOG_S3_ACCESS__KEY__ID: minio
    CATALOG_S3_SECRET__ACCESS__KEY: minio123
    CATALOG_S3_PATH__STYLE__ACCESS: "true"
    AWS_REGION: us-east-1
  ports:
    - "8181:8181"
  depends_on:
    - minio
  networks:
    - trino_network
```

Adicionalmente, se actualiza el depends_on de Trino para incluir iceberg-rest, garantizando que el REST Catalog esté disponible antes de que Trino arranque.

```yaml
▷Run Service
trino:
  image: trinodb/trino:472
  container_name: trino
  hostname: trino
  restart: unless-stopped
  user: root
  environment:
    - TRINO_CONFIG_DIR=/etc/trino
  volumes:
    - ./trino/conf:/etc/trino
    - ./trino/conf/catalog:/etc/trino/catalog
  depends_on:
    - mysql
    - postgres
    - minio
    - iceberg-rest
  ports:
    - "8080:8080"
  networks:
    - trino_network
```

3) Configuración del catálogo de Iceberg

Se crea el fichero *trino/conf/catalog/iceberg.properties*, que configura Trino para usar el REST Catalog como gestor de metadatos y le proporciona las credenciales y endpoint de MinIO para acceder a los ficheros Parquet mediante el protocolo S3 nativo:

```
trino > conf > catalog > iceberg.properties
connector.name=iceberg

iceberg.catalog.type=rest
iceberg.rest-catalog.uri=http://iceberg-rest:8181

iceberg.file-format=PARQUET

fs.native-s3.enabled=true
s3.endpoint=http://minio:9000
s3.region=us-east-1
s3.path-style-access=true
s3.aws-access-key=minio
s3.aws-secret-key=minio123

iceberg.unique-table-location=true
iceberg.register-table-procedure.enabled=true
```

Dos propiedades de este fichero merecen especial atención, ya que fueron necesarias para resolver restricciones específicas de Trino 472:

- *iceberg.unique-table-location=true*: evita conflictos al crear tablas en ubicaciones de MinIO que ya contienen ficheros.
- *iceberg.register-table-procedure.enabled=true*: habilita el procedimiento register_table de Iceberg, necesario para registrar el fichero Parquet existente en MinIO como una tabla Iceberg desde PyIceberg.

4) Verificación

Se reinician los contenedores para aplicar todos los cambios. Se accede al CLI de Trino para verificar que los catálogos están correctamente registrados.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker compose restart trino
time="2026-03-21T19:33:26+01:00" level=warning msg="C:\\Users\\jaime\\Desktop\\BD (Big Data)\\Ecosistema Hadoop\\Teoría\\2025
`version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Restarting 1/1
 ✓ Container trino  Started
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker exec -it trino trino
trino> SHOW CATALOGS;
 Catalog
----------
 iceberg
 mysql
 postgres
 system
(4 rows)

Query 20260321_183338_00000_n3qrn, FINISHED, 1 node
Splits: 19 total, 19 done (100.00%)
0.58 [0 rows, 0B] [0 rows/s, 0B/s]
```

```
trino> SHOW SCHEMAS FROM iceberg;
       Schema
--------------------
 information_schema
(1 row)

Query 20260321_184635_00003_n3qrn, FINISHED, 1 node
Splits: 19 total, 19 done (100.00%)
0.57 [1 rows, 23B] [1 rows/s, 40B/s]
```

3.6. Paso 6: Registro del Parquet en Iceberg

El objetivo de este paso es registrar el fichero *daily_dataset.parquet* almacenado en MinIO como una tabla Iceberg, de forma que Trino pueda consultarla mediante SQL. Para ello se utiliza **PyIceberg**, la librería Python oficial de Apache Iceberg.

El reto de este paso reside en que Trino, por sí solo, no puede leer un fichero Parquet suelto almacenado en MinIO sin que existan metadatos Iceberg asociados. PyIceberg permite generar esos metadatos y registrar el fichero como una tabla Iceberg en el REST Catalog, sin necesidad de mover ni transformar el fichero original.

1) Instalación de dependencias y creación del script de registro

Se instalan las librerías necesarias.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> pip install pyiceberg pyarrow boto3
Collecting pyiceberg
  Downloading pyiceberg-0.11.1-cp312-cp312-win_amd64.whl.metadata (4.9 kB)
Requirement already satisfied: pyarrow in c:\python312\lib\site-packages (17.0.0)
Collecting boto3
  Downloading boto3-1.42.73-py3-none-any.whl.metadata (6.7 kB)
Collecting mmh3<6.0.0,>=4.0.0 (from pyiceberg)
  Downloading mmh3-5.2.1-cp312-cp312-win_amd64.whl.metadata (15 kB)
Requirement already satisfied: requests<3.0.0,>=2.20.0 in c:\python312\lib\site-packages (from pyiceberg) (2.32.3)
Requirement already satisfied: click<9.0.0,>=7.1.1 in c:\python312\lib\site-packages (from pyiceberg) (8.1.7)
Requirement already satisfied: rich<15.0.0,>=10.11.0 in c:\python312\lib\site-packages (from pyiceberg) (13.7.1)
```

Se crea el script src/register_iceberg.py. El script realiza las siguientes operaciones:

1. Se conecta al REST Catalog mediante HTTP, proporcionando las credenciales de acceso a MinIO.
2. Lee el esquema del fichero Parquet local para inferir automáticamente los tipos de datos de cada columna.

3. Crea el namespace default y registra la tabla daily_dataset en el REST Catalog, apuntando al bucket s3://smart-meter/ como ubicación.
4. Registra el fichero *daily_dataset.parquet* como fichero de datos de la tabla mediante add_files.

src > register_iceberg.py > ...

```python
import pyarrow.parquet as pq
from pyiceberg.catalog.rest import RestCatalog

# conectar al REST catalog
catalog = RestCatalog(
    name="iceberg",
    uri="http://localhost:8181",
    warehouse="s3://smart-meter/",
    **{
        "s3.endpoint": "http://localhost:9000",
        "s3.access-key-id": "minio",
        "s3.secret-access-key": "minio123",
        "s3.path-style-access": "true",
        "py-io-impl": "pyiceberg.io.pyarrow.PyArrowFileIO",
    }
)

# leer el esquema del parquet local
schema = pq.read_schema("data/daily_dataset.parquet")
print("Esquema detectado:", schema)

# crear el namespace y la tabla Iceberg
catalog.create_namespace("default")
table = catalog.create_table(
    "default.daily_dataset",
    schema=schema,
    location="s3://smart-meter/",
)

# registrar el parquet existente como fichero de datos
table.add_files(file_paths=["s3://smart-meter/daily_dataset.parquet"])
print("Tabla Iceberg registrada correctamente")
```

2) Verificación desde trino

Se accede al CLI de Trino para comprobar que la tabla es visible y contiene datos.

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker exec -it trino trino
trino> SHOW SCHEMAS FROM iceberg;
       Schema
--------------------
 default
 information_schema
(2 rows)

Query 20260321_200153_00001_m634e, FINISHED, 1 node
Splits: 19 total, 19 done (100.00%)
0.23 [2 rows, 35B] [8 rows/s, 150B/s]

trino> SHOW TABLES FROM iceberg.default;
     Table
---------------
 daily_dataset
(1 row)

Query 20260321_200200_00002_m634e, FINISHED, 1 node
Splits: 19 total, 19 done (100.00%)
0.33 [1 rows, 30B] [3 rows/s, 92B/s]

trino> SELECT COUNT(*) FROM iceberg.default.daily_dataset;
  _col0
---------
 3510433
(1 row)

Query 20260321_200207_00003_m634e, FINISHED, 1 node
Splits: 18 total, 18 done (100.00%)
0.79 [3.51M rows, 0B] [4.43M rows/s, 0B/s]

trino> SELECT * FROM iceberg.default.daily_dataset LIMIT 5;
   lclid   |    day     |   energy_median    |    energy_mean     | energy_max | energy_count |     energy_std     |    energy_sum     | energy_min
-----------+------------+--------------------+--------------------+------------+--------------+--------------------+-------------------+------------
 MAC002453 | 2013-01-06 |             0.0465 | 0.1498541645833332 |      1.061 |           48 | 0.2838619608616172 | 7.192999899999998 |      0.013
 MAC002453 | 2013-01-07 |             0.0465 | 0.1307083312499999 |  1.0549999 |           48 | 0.2483295567625401 | 6.273999899999999 |        0.0
 MAC002453 | 2013-01-08 | 0.0289999999999999 | 0.1329374999999999 |      1.062 |           48 | 0.2655962706616223 | 6.380999999999998 |      0.012
 MAC002453 | 2013-01-09 |             0.0465 | 0.1258333354166666 |      1.054 |           48 | 0.2506057703356652 | 6.040000099999999 |      0.011
 MAC002453 | 2013-01-10 |             0.0285 | 0.1220624999999999 |      1.054 |           48 | 0.2558475615167019 | 5.858999999999996 |        0.0
(5 rows)

Query 20260321_200214_00004_m634e, FINISHED, 1 node
Splits: 21 total, 18 done (85.71%)
0.52 [7 rows, 7.79MiB] [13 rows/s, 15.1MiB/s]
```

El resultado confirma que los **3.510.433 registros** de consumo diario están correctamente registrados en Iceberg y son consultables desde Trino directamente sobre el fichero Parquet almacenado en MinIO.

4. Consultas federadas con Trino

En esta sección se presentan las consultas SQL ejecutadas sobre la arquitectura desplegada. Todas ellas combinan datos de las dos fuentes mediante JOINs federados: la tabla *households* almacenada en PostgreSQL y la tabla *daily_dataset* almacenada en MinIO a través de Iceberg. Todas las consultas están almacenadas en la carpeta *joins_federados/*.

4.1. Consumo diario por hogar

Esta primera consulta es una muestra básica del JOIN federado entre ambas fuentes. Para cada registro de consumo diario se recupera el grupo socioeconómico del hogar correspondiente, uniendo ambas tablas por el identificador LCLid.

joins_federados > 01_join_basico.sql

```sql
SELECT
    d.lclid,
    h.acorn_grouped,
    d.day,
    d.energy_sum
FROM iceberg.default.daily_dataset d
JOIN postgres.public.households h ON d.lclid = h.lclid
LIMIT 10;
```

```
PS C:\Users\jaime\Desktop\BD (Big Data)\Ecosistema Hadoop\Teoría\202526-S2-BigDataSQL> docker exec -it trino trino
trino> SELECT
    ->     d.lclid,
    ->     h.acorn_grouped,
    ->     d.day,
    ->     d.energy_sum
    -> FROM iceberg.default.daily_dataset d
    -> JOIN postgres.public.households h ON d.lclid = h.lclid
    -> LIMIT 10;
   lclid    | acorn_grouped |    day     |     energy_sum
-----------+---------------+------------+--------------------
 MAC002453 | Adversity     | 2013-01-06 |  7.192999899999998
 MAC002453 | Adversity     | 2013-01-07 |  6.273999899999999
 MAC002453 | Adversity     | 2013-01-08 |  6.380999999999998
 MAC002453 | Adversity     | 2013-01-09 |  6.040000099999999
 MAC002453 | Adversity     | 2013-01-10 |  5.858999999999996
 MAC002453 | Adversity     | 2013-01-11 |              6.174
 MAC002453 | Adversity     | 2013-01-12 |              9.318
 MAC002453 | Adversity     | 2013-01-13 |  5.890999999999997
 MAC002453 | Adversity     | 2013-01-14 |  7.619999899999998
 MAC002453 | Adversity     | 2013-01-15 | 7.0410001000000015
(10 rows)
```

Los resultados muestran cómo Trino es capaz de combinar en una única consulta datos provenientes de dos sistemas completamente distintos: PostgreSQL y MinIO. Se puede observar que el hogar MAC002453, clasificado como Adversity, presenta consumos diarios de entre 5 y 9 kWh durante la primera quincena enero de 2013.

4.2. Consumo medio diario por grupo socioeconómico (Acorn)

Esta consulta agrega los datos de consumo por grupo socioeconómico, calculando el consumo medio diario y el número de hogares distintos en cada grupo.

```
joins_federados > 02_consumo_medio_por_acorn.sql
SELECT
    h.acorn_grouped,
    ROUND(AVG(d.energy_sum), 4) AS avg_energy_sum,
    ROUND(AVG(d.energy_mean), 4) AS avg_energy_mean,
    COUNT(DISTINCT d.lclid) AS num_hogares
FROM iceberg.default.daily_dataset d
JOIN postgres.public.households h ON d.lclid = h.lclid
GROUP BY h.acorn_grouped
ORDER BY avg_energy_sum DESC;
```

```
trino> SELECT
    ->     h.acorn_grouped,
    ->     ROUND(AVG(d.energy_sum), 4) AS avg_energy_sum,
    ->     ROUND(AVG(d.energy_mean), 4) AS avg_energy_mean,
    ->     COUNT(DISTINCT d.lclid) AS num_hogares
    -> FROM iceberg.default.daily_dataset d
    -> JOIN postgres.public.households h ON d.lclid = h.lclid
    -> GROUP BY h.acorn_grouped
    -> ORDER BY avg_energy_sum DESC;
 acorn_grouped | avg_energy_sum | avg_energy_mean | num_hogares
---------------+----------------+-----------------+-------------
 ACORN-        |        12.0038 |          0.2512 |           2
 ACORN-U       |        11.6173 |          0.2432 |          49
 Affluent      |        11.4786 |            0.24 |        2192
 Comfortable   |        10.0045 |          0.2092 |        1507
 Adversity     |         8.5034 |          0.1779 |        1816
(5 rows)

Query 20260321_201123_00009_m634e, FINISHED, 1 node
Splits: 104 total, 104 done (100.00%)
0.79 [3.52M rows, 31.7MiB] [4.48M rows/s, 40.3MiB/s]
```

Los resultados revelan una correlación clara entre nivel socioeconómico y consumo energético: los hogares Affluent consumen de media más que los hogares Adversity (11,47 vs 8,50 kWh/día). Este patrón es coherente con lo esperado, ya que los hogares de mayor poder adquisitivo tienden a tener viviendas más grandes y mayor número de electrodomésticos. Los grupos ACORN- y ACORN-U presentan consumos elevados pero una muestra muy reducida (2 y 49 hogares respectivamente), por lo que no son estadísticamente representativos.

4.3. Consumo medio diario por tipo de tarifa (Std vs ToU)

Esta consulta compara el consumo medio entre los hogares con tarifa estándar (Std) y los hogares con tarifa por uso horario (ToU).

joins_federados > 03_consumo_por_tarifa.sql

```sql
SELECT
    h.stdortou,
    ROUND(AVG(d.energy_sum), 4) AS avg_energy_sum,
    ROUND(AVG(d.energy_mean), 4) AS avg_energy_mean,
    COUNT(DISTINCT d.lclid) AS num_hogares
FROM iceberg.default.daily_dataset d
JOIN postgres.public.households h ON d.lclid = h.lclid
GROUP BY h.stdortou
ORDER BY avg_energy_sum DESC;
```

```
trino> SELECT
    ->     h.stdortou,
    ->     ROUND(AVG(d.energy_sum), 4) AS avg_energy_sum,
    ->     ROUND(AVG(d.energy_mean), 4) AS avg_energy_mean,
    ->     COUNT(DISTINCT d.lclid) AS num_hogares
    -> FROM iceberg.default.daily_dataset d
    -> JOIN postgres.public.households h ON d.lclid = h.lclid
    -> GROUP BY h.stdortou
    -> ORDER BY avg_energy_sum DESC;
 stdortou | avg_energy_sum | avg_energy_mean | num_hogares
----------+----------------+-----------------+-------------
 Std      |        10.2816 |           0.215 |        4443
 ToU      |         9.4988 |          0.1986 |        1123
(2 rows)

Query 20260321_201129_00010_m634e, FINISHED, 1 node
Splits: 104 total, 104 done (100.00%)
0.66 [3.52M rows, 31.7MiB] [5.33M rows/s, 47.9MiB/s]
```

Los resultados muestran que los hogares con tarifa estándar (Std) consumen de media más que los de tarifa por uso horario (ToU) (10,28 vs 9,50 kWh/día). Esto tiene una explicación lógica: la tarifa ToU penaliza el consumo en horas punta y lo abarata en horas valle, incentivando a los hogares a desplazar su consumo a franjas horarias más económicas, lo que se traduce en un consumo total más eficiente. Cabe destacar además que el grupo Std tiene casi cuatro veces más hogares que el grupo ToU (4.443 vs 1.123).

4.4. Consumo medio diario por mes y grupo socioeconómico

Esta consulta añade la dimensión temporal al análisis, mostrando cómo varía el consumo medio mensual en función del grupo socioeconómico.

joins_federados > 04_consumo_por_mes_y_acorn.sql

```sql
SELECT
    h.acorn_grouped,
    MONTH(CAST(d.day AS DATE)) AS mes,
    ROUND(AVG(d.energy_sum), 4) AS avg_energy_sum
FROM iceberg.default.daily_dataset d
JOIN postgres.public.households h ON d.lclid = h.lclid
GROUP BY h.acorn_grouped, MONTH(CAST(d.day AS DATE))
ORDER BY h.acorn_grouped, mes;
```

```
trino> SELECT
    ->     h.acorn_grouped,
    ->     MONTH(CAST(d.day AS DATE)) AS mes,
    ->     ROUND(AVG(d.energy_sum), 4) AS avg_energy_sum
    -> FROM iceberg.default.daily_dataset d
    -> JOIN postgres.public.households h ON d.lclid = h.lclid
    -> GROUP BY h.acorn_grouped, MONTH(CAST(d.day AS DATE))
    -> ORDER BY h.acorn_grouped, mes;
 acorn_grouped | mes | avg_energy_sum
---------------+-----+----------------
 ACORN-        |   1 |         8.5726
 ACORN-        |   2 |         8.2036
 ACORN-        |   4 |         6.9434
 ACORN-        |   5 |        10.5173
 ACORN-        |   6 |        12.1097
 ACORN-        |   7 |        11.8192
 ACORN-        |   8 |         11.403
 ACORN-        |   9 |        11.6192
 ACORN-        |  10 |        14.0918
 ACORN-        |  11 |        14.5792
 ACORN-        |  12 |        13.3237
 ACORN-U       |   1 |        14.5612
 ACORN-U       |   2 |         13.827
 ACORN-U       |   3 |        15.2607
 ACORN-U       |   4 |        11.1819
 ACORN-U       |   5 |         9.5864
 ACORN-U       |   6 |         8.6481
 ACORN-U       |   7 |         8.3838
 ACORN-U       |   8 |         9.3549
 ACORN-U       |   9 |        10.2229
 ACORN-U       |  10 |        10.8771
 ACORN-U       |  11 |        12.5197
 ACORN-U       |  12 |        14.1391
 Adversity     |   1 |         9.8964
 Adversity     |   2 |         9.5856
 Adversity     |   3 |         9.7592
 Adversity     |   4 |         8.5858
 Adversity     |   5 |          7.632
 Adversity     |   6 |         7.2696
 Adversity     |   7 |         7.2056
 Adversity     |   8 |         7.0952
 Adversity     |   9 |           7.49
 Adversity     |  10 |         8.2609
```

Los resultados confirman el patrón estacional del consumo energético: todos los grupos socioeconómicos presentan consumos notablemente más altos en los meses de invierno (noviembre, diciembre, enero) que en los meses de verano (junio, julio, agosto), lo cual es coherente con el clima de Londres y el uso de calefacción. Este patrón se mantiene de forma consistente en todos los grupos, aunque con diferente intensidad según el nivel socioeconómico.

4.5. Top 10 hogares con mayor consumo total histórico

Esta consulta identifica los diez hogares con mayor consumo energético acumulado a lo largo de todo el periodo de datos.

```
joins_federados > 05_top10_hogares_mas_consumidores.sql
SELECT
    d.lclid,
    h.acorn_grouped,
    h.stdortou,
    ROUND(SUM(d.energy_sum), 2) AS total_energy,
    COUNT(d.day) AS dias_medidos
FROM iceberg.default.daily_dataset d
JOIN postgres.public.households h ON d.lclid = h.lclid
GROUP BY d.lclid, h.acorn_grouped, h.stdortou
ORDER BY total_energy DESC
LIMIT 10;
```

```
trino> SELECT
    ->     d.lclid,
    ->     h.acorn_grouped,
    ->     h.stdortou,
    ->     ROUND(SUM(d.energy_sum), 2) AS total_energy,
    ->     COUNT(d.day) AS dias_medidos
    -> FROM iceberg.default.daily_dataset d
    -> JOIN postgres.public.households h ON d.lclid = h.lclid
    -> GROUP BY d.lclid, h.acorn_grouped, h.stdortou
    -> ORDER BY total_energy DESC
    -> LIMIT 10;
   lclid    | acorn_grouped | stdortou | total_energy | dias_medidos
-----------+---------------+----------+--------------+--------------
 MAC004179 | Affluent      | Std      |     65623.65 |          649
 MAC005406 | Affluent      | Std      |     51950.87 |          733
 MAC000557 | Affluent      | Std      |      44660.9 |          689
 MAC000985 | Affluent      | ToU      |     43031.15 |          668
 MAC001145 | Affluent      | Std      |     42592.31 |          661
 MAC004993 | Affluent      | Std      |     40768.81 |          764
 MAC000105 | Affluent      | Std      |     40426.19 |          793
 MAC001739 | Comfortable   | Std      |     37827.68 |          638
 MAC004593 | Affluent      | ToU      |     37340.36 |          598
 MAC003329 | Comfortable   | Std      |     36346.25 |          522
(10 rows)

Query 20260321_201823_00013_m634e, FINISHED, 1 node
Splits: 88 total, 88 done (100.00%)
0.74 [3.52M rows, 17.4MiB] [4.77M rows/s, 23.6MiB/s]
```

Los resultados muestran que los diez hogares con mayor consumo histórico pertenecen casi en su totalidad al grupo Affluent, lo cual es coherente con los resultados de la consulta anterior. El hogar MAC004179 destaca de forma notable con un consumo total de 65.623 kWh en 649 días medidos. Es también reseñable que la mayoría de estos hogares tienen tarifa estándar (Std), lo que sugiere que los grandes consumidores no se acogen a tarifas por uso horario.

4.6. Consumo medio en invierno vs verano por grupo socioeconómico

Esta consulta cuantifica la diferencia de consumo entre los meses de invierno (diciembre, enero, febrero) y los meses de verano (junio, julio, agosto) para cada grupo socioeconómico.

```
joins_federados > 06_consumo_verano_vs_invierno.sql
SELECT
    h.acorn_grouped,
    ROUND(AVG(CASE WHEN MONTH(CAST(d.day AS DATE)) IN (12, 1, 2) THEN d.energy_sum END), 4) AS avg_invierno,
    ROUND(AVG(CASE WHEN MONTH(CAST(d.day AS DATE)) IN (6, 7, 8) THEN d.energy_sum END), 4) AS avg_verano,
    ROUND(
        AVG(CASE WHEN MONTH(CAST(d.day AS DATE)) IN (12, 1, 2) THEN d.energy_sum END) -
        AVG(CASE WHEN MONTH(CAST(d.day AS DATE)) IN (6, 7, 8) THEN d.energy_sum END),
    4) AS diferencia
FROM iceberg.default.daily_dataset d
JOIN postgres.public.households h ON d.lclid = h.lclid
GROUP BY h.acorn_grouped
ORDER BY diferencia DESC;
```

```
trino> SELECT
    ->     h.acorn_grouped,
    ->     ROUND(AVG(CASE WHEN MONTH(CAST(d.day AS DATE)) IN (12, 1, 2) THEN d.energy_sum END), 4) AS avg_invierno,
    ->     ROUND(AVG(CASE WHEN MONTH(CAST(d.day AS DATE)) IN (6, 7, 8) THEN d.energy_sum END), 4) AS avg_verano,
    ->     ROUND(
    ->         AVG(CASE WHEN MONTH(CAST(d.day AS DATE)) IN (12, 1, 2) THEN d.energy_sum END) -
    ->         AVG(CASE WHEN MONTH(CAST(d.day AS DATE)) IN (6, 7, 8) THEN d.energy_sum END),
    ->     4) AS diferencia
    -> FROM iceberg.default.daily_dataset d
    -> JOIN postgres.public.households h ON d.lclid = h.lclid
    -> GROUP BY h.acorn_grouped
    -> ORDER BY diferencia DESC;
 acorn_grouped | avg_invierno | avg_verano | diferencia
---------------+--------------+------------+------------
 ACORN-U       |      14.1874 |     8.8075 |     5.3799
 Affluent      |      13.8936 |     8.9222 |     4.9714
 Comfortable   |      11.7834 |     8.1689 |     3.6145
 Adversity     |        9.809 |     7.1875 |     2.6215
 ACORN-        |       11.088 |    11.7737 |    -0.6857
(5 rows)

Query 20260321_202248_00019_m634e, FINISHED, 1 node
Splits: 72 total, 72 done (100.00%)
0.74 [3.52M rows, 17.4MiB] [4.74M rows/s, 23.4MiB/s]
```

Los resultados muestran que todos los grupos, salvo ACORN-, presentan un consumo significativamente mayor en invierno que en verano. El grupo ACORN-U registra la mayor diferencia (5,38 kWh/día), seguido de Affluent (4,97 kWh/día) y Comfortable (3,61 kWh/día). El grupo Adversity presenta la menor diferencia (2,62 kWh/día), lo que podría indicar un menor uso de calefacción, posiblemente por razones económicas. El caso del grupo ACORN- es atípico, mostrando un consumo ligeramente superior en verano, aunque su muestra es demasiado reducida para extraer conclusiones.

4.7. Hogares sin datos de consumo en MinIO

Esta consulta utiliza un LEFT JOIN para identificar hogares registrados en PostgreSQL que no tienen ningún dato de consumo en la tabla Iceberg de MinIO.

```
joins_federados > 07_hogares_sin_datos_consumo.sql
SELECT
    h.lclid,
    h.acorn_grouped,
    h.stdortou
FROM postgres.public.households h
LEFT JOIN iceberg.default.daily_dataset d ON h.lclid = d.lclid
WHERE d.lclid IS NULL;
```

```
trino> SELECT
    ->     h.lclid,
    ->     h.acorn_grouped,
    ->     h.stdortou
    -> FROM postgres.public.households h
    -> LEFT JOIN iceberg.default.daily_dataset d ON h.lclid = d.lclid
    -> WHERE d.lclid IS NULL;
 lclid | acorn_grouped | stdortou
-------+---------------+----------
(0 rows)

Query 20260321_201904_00015_m634e, FINISHED, 1 node
Splits: 39 total, 39 done (100.00%)
0.85 [3.52M rows, 242KiB] [4.14M rows/s, 285KiB/s]
```

El resultado de 0 filas indica que todos los hogares registrados en PostgreSQL tienen al menos un registro de consumo en MinIO, lo que confirma la integridad referencial entre ambas fuentes de datos a pesar de estar almacenadas en sistemas completamente distintos.

4.8. Variabilidad de consumo diario por grupo socioeconómico

Esta última consulta analiza la irregularidad del consumo energético dentro de cada día, usando la desviación estándar, el rango y el pico máximo de consumo como indicadores de variabilidad.

joins_federados > 08_variabilidad_consumo_por_acorn.sql

```sql
SELECT
    h.acorn_grouped,
    ROUND(AVG(d.energy_std), 4) AS avg_desviacion,
    ROUND(AVG(d.energy_max - d.energy_min), 4) AS avg_rango,
    ROUND(AVG(d.energy_max), 4) AS avg_pico_maximo
FROM iceberg.default.daily_dataset d
JOIN postgres.public.households h ON d.lclid = h.lclid
GROUP BY h.acorn_grouped
ORDER BY avg_desviacion DESC;
```

```
trino> SELECT
    ->     h.acorn_grouped,
    ->     ROUND(AVG(d.energy_std), 4) AS avg_desviacion,
    ->     ROUND(AVG(d.energy_max - d.energy_min), 4) AS avg_rango,
    ->     ROUND(AVG(d.energy_max), 4) AS avg_pico_maximo
    -> FROM iceberg.default.daily_dataset d
    -> JOIN postgres.public.households h ON d.lclid = h.lclid
    -> GROUP BY h.acorn_grouped
    -> ORDER BY avg_desviacion DESC;
 acorn_grouped | avg_desviacion | avg_rango | avg_pico_maximo
---------------+----------------+-----------+-----------------
 ACORN-U       |         0.2068 |    0.8582 |          0.9165
 ACORN-        |         0.2008 |    0.9109 |          0.9951
 Affluent      |         0.1926 |    0.8398 |          0.9094
 Comfortable   |         0.1685 |    0.7663 |          0.8253
 Adversity     |         0.1505 |    0.6993 |           0.747
(5 rows)

Query 20260321_201917_00016_m634e, FINISHED, 1 node
Splits: 72 total, 72 done (100.00%)
0.70 [3.52M rows, 38MiB] [5.04M rows/s, 54.3MiB/s]
```

Los resultados muestran que los hogares Affluent presentan una mayor variabilidad de consumo a lo largo del día que los hogares Adversity (desviación de 0,19 vs 0,15). Esto sugiere que los hogares de mayor nivel socioeconómico tienen hábitos de consumo más irregulares, con mayores picos y valles a lo largo del día, posiblemente asociados al uso de electrodomésticos de alta potencia como lavavajillas, secadoras o sistemas de climatización. Los hogares Adversity, en cambio, presentan un consumo más uniforme y contenido a lo largo del día.

5. Conclusiones

En esta sección se reflexiona sobre el proceso seguido, los obstáculos encontrados y las decisiones tomadas para superarlos, con el objetivo de documentar las lecciones aprendidas más relevantes del entregable.

5.1. Decisiones de arquitectura: de Hive a Iceberg

La primera dificultad significativa surgió al intentar conectar Trino con MinIO a través del catálogo Hive. El enfoque inicial consistía en utilizar el **Hive Metastore** como gestor de metadatos para las tablas almacenadas en MinIO, pero al intentar registrar el fichero Parquet mediante una ruta *s3://*, el Hive Metastore devolvía el error "*UnsupportedFileSystemException: No FileSystem for scheme "s3'"*", lo que impedía que reconociera MinIO como sistema de ficheros válido.

Tras consultar con el profesor, se descartó el uso de Hive Metastore y se optó por **Apache Iceberg con el REST Catalog**, una alternativa más moderna y compatible con el stack desplegado. Esta decisión fue clave para el resto del entregable, ya que Iceberg está diseñado desde su origen para trabajar con almacenamientos de objetos como S3 y MinIO, evitando las incompatibilidades propias de Hive.

5.2. Compatibilidad de Trino 472 con Iceberg

Una vez adoptado Iceberg, surgió un segundo obstáculo: la configuración inicial del catálogo Iceberg usaba ***iceberg.catalog.type=filesystem***, un valor que existía en versiones anteriores de Trino pero que **había sido eliminado en la versión 472**. Esto provocaba que Trino no arrancara correctamente al cargar el catálogo.

La solución fue cambiar al tipo **rest**, configurando el **Iceberg REST Catalog como servicio independiente dentro del docker-compose.yml**. Adicionalmente, fue necesario añadir dos propiedades específicas al fichero *iceberg.properties* para evitar restricciones propias de Trino 472: *iceberg.unique-table-location=true* e *iceberg.register-table-procedure.enabled=true*.

5.3. Registro del Parquet en Iceberg

El tercer obstáculo, y el más laborioso, fue conseguir que Trino pudiera leer el fichero Parquet almacenado en MinIO a través de Iceberg. El error fundamental es que **Trino no puede leer un fichero Parquet suelto sin metadatos Iceberg asociados**, ya que Iceberg necesita sus propios ficheros de metadatos (en formato JSON) para saber cómo está estructurada la tabla y dónde están los datos.

Se intentaron varias aproximaciones antes de encontrar la solución correcta: crear *tablas staging* con ubicaciones específicas en MinIO, usar el procedimiento *register_table* de Iceberg, y usar *add_files*. Todas ellas fallaron por distintas razones, bien porque Iceberg rechazaba ubicaciones no vacías, bien porque los procedimientos estaban deshabilitados por defecto, o bien porque el fichero Parquet carecía de los metadatos que Iceberg esperaba encontrar.

La solución definitiva fue utilizar **PyIceberg** para generar los metadatos Iceberg directamente sobre el fichero Parquet existente en MinIO, sin necesidad de moverlo ni transformarlo. PyIceberg actúa como cliente del REST Catalog, lee el esquema del Parquet, crea la tabla en el catálogo y registra el fichero como fichero de datos mediante *add_files*.

5.4. Reflexión final

A pesar de las dificultades encontradas, el resultado final es una arquitectura funcional y coherente con los objetivos del entregable: Trino es capaz de ejecutar JOINs federados entre PostgreSQL y MinIO de forma transparente, combinando datos de dos sistemas completamente distintos mediante SQL estándar. Las consultas realizadas han permitido extraer conclusiones interesantes sobre los patrones de consumo energético de los hogares londinenses, demostrando el potencial de este tipo de arquitecturas para el análisis de datos distribuido a gran escala.

El proceso ha puesto de manifiesto que en proyectos de Big Data la configuración del entorno y la integración entre componentes es frecuentemente la parte más compleja y costosa, y que la elección de las tecnologías correctas desde el principio es fundamental para evitar problemas de compatibilidad difíciles de resolver.